SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing COURSE CODE: CSA3712

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow.

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

1. ATM Withdrawal Module – Reverse Engineering

Reverse engineering means understanding a system by observing its inputs, outputs, and error messages. An ATM withdrawal module includes card validation, PIN verification, balance checking, withdrawal limit checking, and cash dispensing. ECP divides inputs into valid and invalid groups, while BVA checks values near minimum and maximum limits.

Test case	Input or condition	Expected result
Valid withdrawal	Amount within limits and balance available	Dispense cash
Below minimum	Amount less than minimum	Reject
Minimum amount	Minimum allowed amount	Accept
Maximum amount	Maximum allowed amount	Accept
Above maximum	Amount exceeds limit	Reject
Insufficient balance	Amount greater than balance	Reject
Invalid PIN	Incorrect PIN	Display error
Invalid amount	Zero or negative amount	Reject

Observed behavior suggests black-box testing. Testing only successful withdrawals is insufficient; invalid inputs and boundary values must also be tested.

Conclusion: ECP and BVA provide systematic coverage and help detect ATM withdrawal defects.

2. Software Module – Control Flow Graph and Path Testing

Reverse engineering the white-box approach involves studying execution traces and code logic to identify decisions, loops, and independent paths. A Control Flow Graph (CFG) represents program flow using nodes and edges. Consider a simple module that validates an input and processes it.

Program condition	Possible paths
Input validation	Valid / Invalid
Processing condition	True / False
Loop condition	Continue / Exit

A suitable strategy is to construct a CFG, identify decision points, calculate Cyclomatic Complexity, and test each independent path. Statement coverage alone is insufficient because it may miss decision outcomes.

Conclusion: Path testing and branch coverage help identify missing execution paths and improve software reliability.

3. Web Application – Login and Transaction System

Answer: Login and transaction components include authentication, transaction validation, balance checking, and transaction processing. Defect logs can suggest which techniques were likely used, but cannot prove the exact original technique.

Component	Test cases	Expected result
Login	Valid credentials	Login successful
Login	Invalid credentials	Error displayed
Password	Empty password	Reject
Transaction	Valid amount and sufficient balance	Transaction successful
Transaction	Amount exceeds balance	Reject
Transaction limits	Minimum and maximum values	Apply limit rules
Processing logic	Success and failure paths	Correct status returned

ECP and BVA should test input groups and boundaries. White-box path testing should examine authentication and transaction decision paths.

Conclusion: Integrating black-box and white-box testing improves coverage and helps prevent login and transaction failures.

4. Inventory Management – Stock Updates and Low-Stock Alerts

Successful and failed transaction logs can help reconstruct inventory business rules. The main components are stock input validation, stock updating, low-stock checking, and alert generation.

Test case	Input or condition	Expected result
Normal stock update	Valid quantity	Stock updated
Zero quantity	No stock change requested	Follow defined business rule
Negative quantity	Invalid quantity	Reject
Stock reaches threshold	Quantity equals alert limit	Apply alert rule
Stock below threshold	Quantity below alert limit	Generate alert
Stock above threshold	Quantity above alert limit	No low-stock alert
Invalid product ID	Unknown product	Display error

ECP separates valid and invalid quantities, while BVA tests values around the low-stock threshold.

Conclusion: Analyzing transaction logs and testing boundaries helps reconstruct hidden rules and detect inventory errors.

5. Library Management Module – CPU and Memory Spikes

CPU and memory profiles can indicate loops, repeated searches, and resource-intensive processing. Performance spikes alone cannot confirm the exact source-code structure. The module can be divided into book search, issue processing, return processing, and record updating.

Observed behavior	Possible cause	Test approach
High CPU during search	Repeated search operations	Test different search sizes
Memory increases with records	Large data structures	Test memory usage
Delay during book issue	Repeated validation	Test independent paths
Slow processing for large data	Inefficient loops	Stress testing
Program fails under load	Resource exhaustion	Load and stability testing

Conceptual CFG: Start → Validate Request → Search Book → Book Found? → Process Issue / Display Error → End. Test valid requests, missing books, large datasets, and repeated operations.

Conclusion: Path testing combined with stress testing helps identify resource-intensive paths and improves system stability.

6. Mobile Payment Gateway – State Transition Testing

A mobile payment gateway processes transactions through states such as Pending, Success, Reversal, and Timeout. State Transition Testing examines how the system moves between states after an event.

Current state	Event	Expected state
Pending	Payment confirmed	Success
Pending	Response timeout	Timeout
Pending	Payment cancelled or failed	Reversal, if applicable
Success	Duplicate payment response	Remain Success; avoid duplicate charge
Timeout	Late success response	Reconcile transaction safely
Reversal	Reversal confirmed	Reversal completed

Conceptual transitions: Pending → Success; Pending → Timeout; Pending → Reversal. Test legal and illegal transitions and verify transaction IDs, amounts, and status consistency.

Conclusion: State Transition Testing prevents duplicate payments, incorrect reversals, and inconsistent transaction statuses.

7. Encrypted Communication Module – Fuzzing and BVA

The encrypted communication module accepts input parameters and produces encrypted outputs. Fuzzing provides unusual, malformed, or unexpected inputs to identify validation and error-handling weaknesses. The module includes input validation, data processing, encryption, and error handling.

Test case	Input	Expected result
Valid input	Correctly formatted data	Encrypted output generated
Empty input	No data	Handle according to specification
Minimum length	Smallest valid input	Accept
Maximum length	Largest permitted input	Accept
Above maximum	Oversized input	Reject safely
Invalid characters	Malformed input	Reject safely
Malformed encoded data	Invalid encoding	Error handled securely

BVA tests input-length boundaries, while fuzzing tests unexpected combinations. Input validation and memory-safe processing help prevent buffer overflows and injection vulnerabilities.

Conclusion: Combining BVA, fuzzing, and security testing improves input validation and protects the communication module.

8. Insurance Premium Calculator – Statement Coverage Defects

The calculator achieved 100% statement coverage but still missed defects in multi-condition if statements. Statement coverage alone is insufficient. A premium decision may depend on age, policy type, and claim history.

Technique	What it checks
Statement coverage	Every statement executes
Branch coverage	True and false decision outcomes
Condition coverage	Each Boolean condition is true and false
Path testing	Independent execution paths

Example test cases are shown below. The exact premium must follow the application's business rules.

Conclusion: Improved branch and condition testing detects calculation defects missed by statement coverage.

Age condition	Policy valid	Claim history	Expected action
Eligible	Yes	No claims	Calculate eligible premium
Eligible	No	No claims	Reject invalid policy
Ineligible	Yes	No claims	Apply eligibility rule
Eligible	Yes	Claims present	Apply claim-related rule

The team should construct a CFG and test independent paths, branches, and important condition combinations. Full path coverage may be impractical for complex logic.

9. Cloud File Synchronization – Lost Files

Lost-file defects may occur because of incorrect handling of file sizes, file types, network interruptions, or synchronization conflicts. User activity logs help identify patterns but may not reveal the exact internal algorithm. ECP divides inputs into valid and invalid groups, and BVA checks file sizes near supported limits.

Test case	Condition	Expected result
Small file	Valid file size	Synchronize successfully
Empty file	Zero-byte file	Handle according to specification
Maximum-size file	Supported maximum	Synchronize successfully
Oversized file	Above maximum	Reject or report clearly
Supported file type	Valid format	Synchronize
Unsupported file type	Invalid format	Reject or report error
Slow network	Delayed connection	Retry safely
Network interruption	Connection lost	Resume without data loss

Stress testing should cover large numbers of files, repeated synchronization, and network failures. Checksums and file-integrity verification should confirm synchronized files remain unchanged.

Conclusion: ECP, BVA, stress testing, and recovery testing reduce synchronization errors and prevent data loss.

10. E-commerce Search and Filter – Decision Table Testing

Answer: An e-commerce search module uses conditions such as category, price, availability, and keywords. SQL query parameters and returned datasets can help reconstruct filtering logic. AND requires all specified conditions to be true, OR requires at least one condition to be true, and NOT excludes matching results.

Test case	A: Category	B: Price	C: In stock	Expected result
1	True	True	True	Include product
2	True	True	False	Exclude if availability is required
3	True	False	True	Exclude product
4	False	True	True	Exclude product
5	True	True	Not selected	Apply remaining filters
6	False	False	False	No matching product

For an AND search, all selected conditions must match. For an OR search, at least one condition must match. NOT excludes products matching the exclusion condition. Test filter combinations, empty results, invalid parameters, and boundary prices. For three Boolean conditions, eight combinations exist, so the full truth table can be used when relevant.

Conclusion: Decision Table Testing provides structured coverage of filtering logic and helps detect incorrect search results.

Code of Conduct:

I ____C.Prasanna(192371035)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: C Prasanna

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation